Inverse Kinematics

1. Function Description

After the program runs, you can request IK or FK services through the terminal to obtain the end-effector pose of the robotic arm and the target robotic arm posture (servo values 1-5).

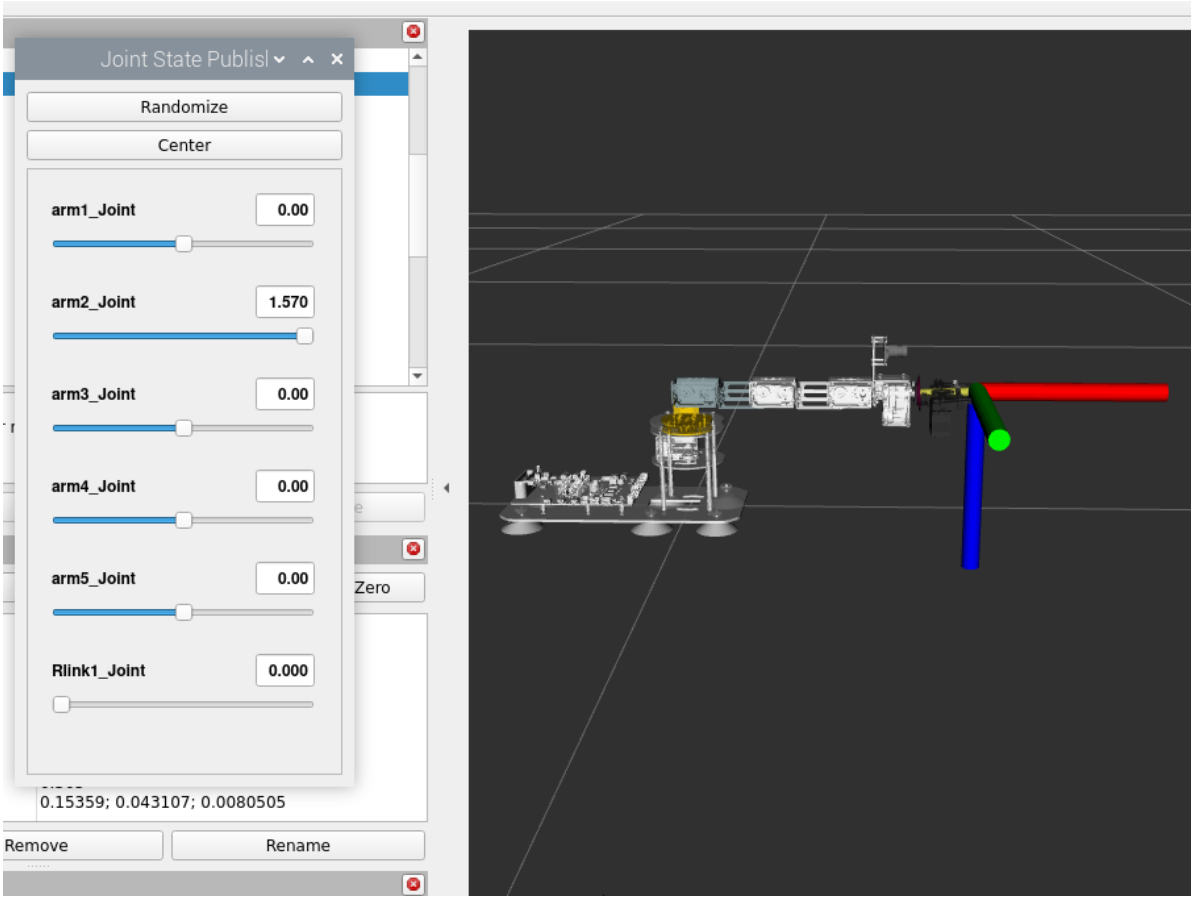
2. Function Startup

After entering the docker container, enter the following commands in the docker terminal:

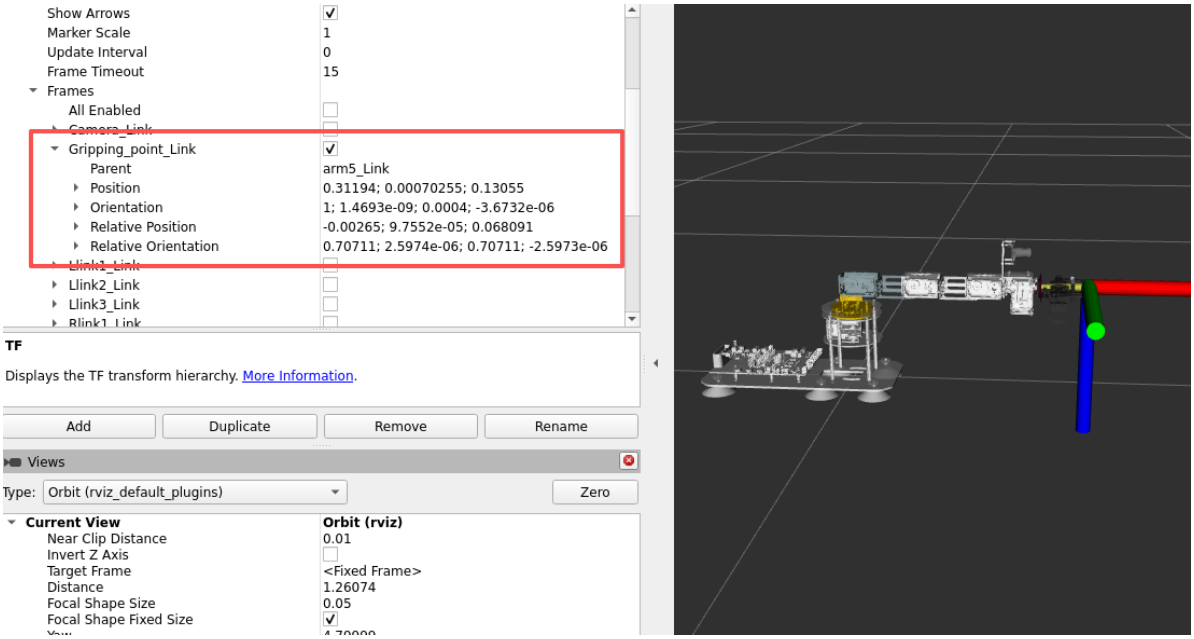
```
#Robotic arm URDF model file display program
ros2 launch dofbot_urdf display.launch.py
#Inverse kinematics service program
ros2 run dofbot_info kinemarics_dofbot
```

2.1. Calling FK

First, move the robotic arm to the downward straight posture. In the RViz joints controller, set it as shown in the figure below to control the RViz model to move to the downward straight posture:



Then, we add TF display, select Fixed_Frame as base_link, then expand the display of **Gripping_point_Link** in TF, and view the Position and Orientation values as shown below:



Next, call FK to verify what the end-effector pose is when the robotic arm is given the downward straight posture. After entering the docker container, enter the following command in the docker terminal:

```
ros2 service call /dofbot_kinemarics dofbot_interface/srv/Kinemarics "{tar_x:
0.0, tar_y: 0.0, tar_z: 0.0, roll: 0.0, pitch: 0.0, yaw: 0.0, cur_joint1: 90.0,
cur_joint2: 180.0, cur_joint3: 90.0, cur_joint4: 90.0, cur_joint5: 90.0,
cur_joint6: 90.0,
kin_name: 'fk'}
```

```
root@jetson-desktop:~# ros2 service call /dofbot_kinemarics dofbot_interface/srv/Kinemarics "{tar_x:
0.0, tar_y: 0.0, tar_z: 0.0, roll: 0.0, pitch: 0.0, yaw: 0.0, cur_joint1: 90.0,
cur_joint2: 180.0, cur_joint3: 90.0, cur_joint4: 90.0, cur_joint5: 90.0,
cur_joint6: 90.0,
kin_name: 'fk'}"
waiting for service to become available...
requester: making request: dofbot_interface.srv.Kinemarics_Request(tar_x=0.0, tar_y=0.0, tar_z=0.0, roll=0.0, pitch=0.0, yaw=0.0, cur_jo
int1=90.0, cur_joint2=180.0, cur_joint3=90.0, cur_joint4=90.0, cur_joint5=90.0, cur_joint6=90.0, kin_name='fk')

response:
dofbot_interface.srv.Kinemarics_Response(joint1=0.0, joint2=0.0, joint3=0.0, joint4=0.0, joint5=0.0, joint6=0.0, x=0.3119399999415937, y
=0.0007025520000000001, z=0.13030000379567486, roll=-3.1415853071795867, pitch=-3.6853730679001116e-06, yaw=-4.852469282682178e-22)
```

As you can see, the xyz and rpy values are basically consistent with those in RViz.

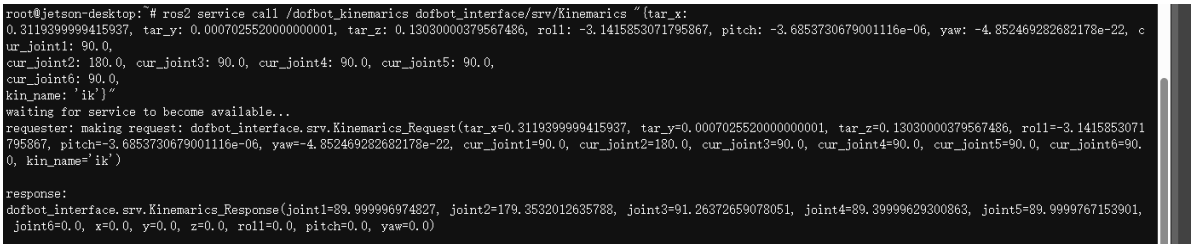
2.2. Calling IK

Based on the end-effector information we just queried, we call IK inverse kinematics to calculate what the values of the six servos are at this end-effector pose. Enter in the terminal:

```
ros2 service call /dofbot_kinemarics dofbot_interface/srv/Kinemarics "{tar_x:
0.3119399999415937, tar_y: 0.0007025520000000001, tar_z: 0.13030000379567486,
roll: -3.1415853071795867, pitch: -3.6853730679001116e-06, yaw:
-4.852469282682178e-22, cur_joint1: 90.0,

cur_joint2: 180.0, cur_joint3: 90.0, cur_joint4: 90.0, cur_joint5: 90.0,

cur_joint6: 90.0,
kin_name: 'ik'}"
waiting for service to become available...
requester: making request:
dofbot_interface.srv.Kinemarics_Request(tar_x=0.3119399999415937,
tar_y=0.0007025520000000001, tar_z=0.13030000379567486, roll=-3.1415853071795867,
pitch=-3.6853730679001116e-06, yaw=-4.852469282682178e-22, cur_joint1=90.0,
cur_joint2=180.0, cur_joint3=90.0, cur_joint4=90.0, cur_joint5=90.0,
cur_joint6=90.0, kin_name='ik')
```



The obtained joint values are basically consistent with the cur_joints given in the FK earlier.